

PROGRAMA DE RESIDÊNCIA EM INTELIGÊNCIA ARTIFICIAL
UNISENAI — 2026

RELATÓRIO TÉCNICO

Detecção de Trincas e Fissuras em Paredes

Sistema de Inspeção Automatizada com Segmentação de Instância

DESAFIO 2

Autor

Daniel Tavares de França

29 de maio de 2026

Aplicação publicada em:

huggingface.co/spaces/DanielTF/detector-trincas

Sumário

1. Introdução.....	3
2. Contextualização do Problema	3
2.1. Descrição do Problema	3
2.2. Solução Proposta	3
3. Dataset	4
3.1. Descrição dos Dados	4
3.2. Formato das Anotações	4
3.3. Preparação dos Dados	4
4. Metodologia	4
4.1. Arquitetura do Modelo	4
4.2. Estratégia de Treinamento	5
4.3. Ambiente de Desenvolvimento	5
5. Resultados	5
5.1. Métricas de Avaliação	5
5.2. Análise das Métricas	6
5.3. Análise Visual das Predições	6
6. Aplicação Web.....	7
6.1. Funcionalidades	7
6.2. Acesso	7
7. Discussão	7
7.1. Pontos Fortes.....	7
7.2. Limitações e Pontos de Melhoria.....	7
7.3. Trabalhos Futuros	7
8. Conclusão.....	8
9. Referências.....	8

1. Introdução

O setor da construção civil enfrenta desafios constantes relacionados à qualidade das obras entregues. Uma das etapas mais críticas do processo construtivo é a inspeção das paredes após a concretagem, momento em que trincas e fissuras precisam ser identificadas e corrigidas antes da aplicação das camadas de revestimento ou de pintura.

Atualmente, essa inspeção é realizada de forma manual por profissionais especializados. A fadiga acumulada ao longo da jornada de trabalho, somada à subjetividade da avaliação visual, pode comprometer a atenção dos inspetores e resultar em falhas que passam despercebidas. Quando não corrigidas a tempo, essas falhas geram retrabalho nas etapas seguintes, elevam os custos da obra e impactam negativamente a satisfação dos clientes.

Diante desse cenário, o presente projeto propõe o desenvolvimento de um sistema automatizado de detecção de trincas e fissuras em paredes, baseado em técnicas de visão computacional e aprendizagem profunda. A solução utiliza o modelo YOLOv8n-seg para realizar a segmentação de instância das fissuras, identificando não apenas a presença, mas também a localização e o contorno de cada anomalia detectada.

O modelo foi concebido com tamanho reduzido, de modo a viabilizar sua execução em dispositivos de baixo custo, como os smartphones já utilizados pelas equipes de campo, favorecendo a adoção da ferramenta sem a necessidade de investimentos adicionais em hardware.

2. Contextualização do Problema

2.1. Descrição do Problema

Ao final do processo de concretagem, cada parede levantada deve ser inspecionada em busca de trincas e fissuras que possam comprometer a integridade estrutural ou a qualidade do acabamento superficial. O problema central identificado é a dependência da inspeção humana, sujeita a variabilidade e a fadiga.

As consequências diretas de falhas nessa inspeção incluem:

- Retrabalho nas etapas de revestimento e pintura;
- Aumento dos custos de manutenção após a entrega;
- Redução da satisfação dos clientes finais;
- Possíveis problemas estruturais de médio e longo prazo.

2.2. Solução Proposta

O sistema desenvolvido automatiza o processo de inspeção por meio de um modelo de inteligência artificial executável em dispositivo móvel. O operador fotografa a parede inspecionada e o sistema retorna a localização das trincas detectadas, acompanhada de um índice de confiança para cada detecção.

A abordagem de segmentação de instância foi escolhida em detrimento da simples classificação ou da detecção por caixa delimitadora, pois permite mapear o contorno de cada fissura, fornecendo informação geométrica mais rica para subsidiar a decisão de reparo.

3. Dataset

3.1. Descrição dos Dados

O conjunto de dados utilizado no desenvolvimento do modelo é composto por 1.551 imagens de paredes, acompanhadas de 1.551 arquivos de anotação correspondentes, no formato de segmentação do YOLO. Cada imagem possui um arquivo de rótulos associado, e o problema foi modelado com uma única classe de objeto: trinca.

Parâmetro	Valor	Observação
Total de imagens	1.551	Conjunto completo
Conjunto de treino	1.240 (80%)	Divisão aleatória
Conjunto de validação	311 (20%)	423 instâncias anotadas
Número de classes	1 (trinca)	Classe única
Formato das anotações	YOLO (segmentação)	Polígonos normalizados
Resolução de treino	640 × 640 px	Redimensionamento automático

Tabela 1 — Composição do conjunto de dados.

3.2. Formato das Anotações

As anotações seguem o formato de segmentação de instância do YOLO. Cada linha de um arquivo de rótulos inicia com o índice da classe e é seguida por uma sequência de pares de coordenadas (x, y) normalizadas entre 0 e 1, que descrevem o polígono de contorno de uma fissura. Esse formato permite treinar o modelo para reconhecer tanto a presença quanto a forma de cada trinca.

Estrutura de cada linha: <índice_da_classe> x_1 y_1 x_2 y_2 ... x_n y_n

3.3. Preparação dos Dados

Antes do treinamento, foi realizada a verificação de correspondência entre os arquivos de imagem e de anotação, confirmando que os 1.551 pares estavam completos e íntegros. Em seguida, o conjunto foi dividido aleatoriamente na proporção de 80% para treino e 20% para validação, utilizando semente aleatória fixa (seed = 42) para garantir a reprodutibilidade da divisão.

Os arquivos foram organizados na estrutura de diretórios esperada pelo framework e descritos em um arquivo de configuração (data.yaml) com uma classe nomeada como “trinca”.

4. Metodologia

4.1. Arquitetura do Modelo

O modelo selecionado foi o YOLOv8n-seg (You Only Look Once — versão nano com segmentação de instância), desenvolvido pela Ultralytics. A família YOLO é amplamente reconhecida na literatura por combinar boa acurácia com baixo custo computacional em tarefas de detecção em tempo real.

Em sua forma final (camadas fundidas), o modelo possui 3.258.259 parâmetros, 11,3 GFLOPs e 6,8 MB em disco. A versão nano foi escolhida pelos seguintes motivos:

- Tamanho compacto (6,8 MB), adequado a dispositivos móveis com armazenamento limitado;
- Baixo consumo de memória em inferência, compatível com aparelhos de entrada;
- Velocidade de inferência elevada, que favorece a análise em tempo real;
- Desempenho adequado mesmo com um conjunto de dados de tamanho moderado.

4.2. Estratégia de Treinamento

O treinamento empregou a técnica de transferência de aprendizado (transfer learning), partindo dos pesos pré-treinados do YOLOv8n-seg no conjunto COCO. Essa abordagem reaproveita representações visuais já aprendidas em grande volume de imagens, reduzindo o tempo de convergência e a quantidade de dados necessária para obter bons resultados.

Parâmetro	Valor
Modelo base	YOLOv8n-seg (pré-treinado em COCO)
Épocas	50 (concluídas; sem parada antecipada)
Tamanho do lote (batch)	16
Resolução de entrada	640 × 640 px
Otimizador	AdamW (selecionado automaticamente; lr = 0,002; momentum = 0,9)
Parada antecipada	Paciência de 15 épocas (não acionada)
Hardware	GPU NVIDIA Tesla T4 (~15 GB)
Tempo total de treinamento	0,818 h (≈ 49 minutos)
Framework	Ultralytics 8.4.57 / PyTorch 2.11 / Python 3.12

Tabela 2 — Configuração do treinamento. O otimizador foi definido em modo automático pelo framework, que selecionou o AdamW.

4.3. Ambiente de Desenvolvimento

O treinamento foi executado no Google Colaboratory, utilizando uma GPU NVIDIA Tesla T4 disponibilizada pela plataforma. O ambiente permitiu o treinamento completo das 50 épocas em aproximadamente 49 minutos.

5. Resultados

5.1. Métricas de Avaliação

Após o treinamento, o melhor checkpoint do modelo foi avaliado no conjunto de validação, composto por 311 imagens (423 instâncias de trinca) não vistas durante o treinamento. Por se tratar de uma tarefa de segmentação de instância, as métricas de máscara são consideradas as principais; as métricas de detecção por caixa delimitadora são apresentadas como complemento.

Métrica	Máscara	Caixa	Avaliação
Precisão	76,4%	80,0%	Boa
Recall (revocação)	59,1%	66,4%	Satisfatória
mAP@50	62,3%	73,4%	Satisfatória
mAP@50–95	22,6%	55,2%	A melhorar

Tabela 3 — Métricas no conjunto de validação (311 imagens, 423 instâncias). “Máscara” refere-se à segmentação; “Caixa”, à detecção por caixa delimitadora.

Eficiência	Valor	Referência
Tamanho do modelo	6,8 MB	≤ 50 MB
Tempo de inferência	5,5 ms/imagem	≤ 100 ms
Tempo total por imagem	≈ 13,4 ms	pré + inferência + pós

Tabela 4 — Indicadores de eficiência medidos na GPU Tesla T4.

5.2. Análise das Métricas

A precisão de 76,4% na segmentação indica que cerca de três quartos das detecções realizadas correspondem a fissuras reais, o que reflete uma taxa de falsos positivos relativamente baixa e reduz a necessidade de revisões manuais de alertas incorretos.

O recall de 59,1% indica que o modelo identifica aproximadamente 59% das fissuras presentes nas imagens. Há espaço para melhoria nesse indicador, sobretudo no que se refere a fissuras de menor dimensão, que tendem a ser as mais difíceis de detectar — tanto para o modelo quanto para inspetores humanos.

O mAP@50 de 62,3% mostra que o modelo localiza as trincas de forma consistente quando se admite uma sobreposição (IoU) de pelo menos 0,5 com a anotação de referência. Já o mAP@50–95 de 22,6% revela que, sob limiares de IoU mais rigorosos, a aderência do contorno das máscaras ainda é limitada: o modelo localiza bem a região da fissura, mas o traçado exato do contorno pode ser refinado. Como esperado, as métricas de caixa delimitadora são superiores às de máscara, pois delimitar a região de uma trinca é uma tarefa mais simples do que reproduzir seu contorno.

Em relação à eficiência, o tamanho de 6,8 MB e o tempo de inferência de poucos milissegundos por imagem confirmam a viabilidade de execução em tempo real, inclusive em hardware limitado.

5.3. Análise Visual das Predições

A inspeção visual das predições sobre o conjunto de validação mostra que o modelo identifica fissuras de diferentes orientações — horizontais, verticais e diagonais — e segmenta de forma independente cada instância presente em uma mesma imagem. As predições foram geradas no

próprio notebook, com limiar de confiança de 0,25, e os contornos previstos acompanham, de modo geral, a geometria das trincas anotadas.

6. Aplicação Web

O modelo treinado foi integrado a uma aplicação web interativa, desenvolvida com a biblioteca Gradio e publicada na plataforma Hugging Face Spaces.

6.1. Funcionalidades

- Envio de uma imagem de parede diretamente pelo navegador;
- Ajuste do limiar de confiança da detecção;
- Exibição da imagem com os contornos de segmentação das trincas sobrepostos;
- Interface acessível por qualquer dispositivo com navegador web.

6.2. Acesso

A aplicação está disponível de forma pública e gratuita, sem necessidade de instalação ou configuração por parte do usuário final.

URL pública: huggingface.co/spaces/DanielTF/detector-trincas

7. Discussão

7.1. Pontos Fortes

- A escolha do YOLOv8n-seg equilibrou desempenho e eficiência computacional;
- A segmentação de instância agrega informação geométrica das fissuras, indo além da simples detecção;
- O modelo de apenas 6,8 MB atende ao requisito de execução em dispositivos móveis;
- A velocidade de inferência da ordem de poucos milissegundos viabiliza aplicações em tempo real;
- A publicação na Hugging Face Spaces garante acesso público e permanente à solução.

7.2. Limitações e Pontos de Melhoria

- O recall de 59,1% indica que cerca de 41% das fissuras presentes não são detectadas, sobretudo as de menor dimensão;
- O mAP@50–95 de 22,6% na segmentação revela que o contorno das máscaras ainda é pouco preciso sob limiares de IoU rigorosos;
- O conjunto de 1.551 imagens, embora útil, é de tamanho moderado; um volume maior tende a melhorar as métricas;
- Os resultados foram obtidos em conjunto de validação; o desempenho ainda não foi verificado em condições reais de campo.

7.3. Trabalhos Futuros

- Ampliar o conjunto de dados, incluindo diferentes condições de iluminação, distância e tipos de superfície;
- Avaliar modelos de maior capacidade (YOLOv8s-seg, YOLOv8m-seg) em cenários com hardware mais robusto;
- Refinar a qualidade das máscaras para elevar o mAP@50–95;
- Implementar a classificação de severidade das fissuras (leve, moderada e grave);
- Realizar validação em campo com inspeção humana paralela, a fim de medir o ganho real de qualidade;
- Integrar o sistema a uma plataforma de gestão de obras, com histórico e rastreabilidade das inspeções.

8. Conclusão

O projeto demonstrou a viabilidade de desenvolver um sistema automatizado de detecção de trincas e fissuras em paredes utilizando técnicas modernas de visão computacional e aprendizagem profunda, com recursos computacionais acessíveis.

O modelo YOLOv8n-seg, treinado por 50 épocas em aproximadamente 49 minutos, atingiu, na segmentação, precisão de 76,4%, recall de 59,1%, mAP@50 de 62,3% e mAP@50–95 de 22,6%, com tamanho de apenas 6,8 MB e tempo de inferência da ordem de 5,5 ms por imagem — características compatíveis com a execução em dispositivos de baixo custo.

Os resultados configuram uma prova de conceito promissora. As principais oportunidades de evolução concentram-se no aumento do recall e no refinamento do contorno das máscaras, além da validação em campo. A entrega na forma de aplicação web pública amplia a utilidade da solução tanto para demonstração quanto para uso pelas equipes de inspeção.

9. Referências

- JOCHER, G. et al. Ultralytics YOLOv8. Disponível em: <https://github.com/ultralytics/ultralytics>. Acesso em: maio de 2026.
- REDMON, J. et al. You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- HE, K. et al. Mask R-CNN. Proceedings of the IEEE International Conference on Computer Vision (ICCV), 2017.
- LIN, T.-Y. et al. Microsoft COCO: Common Objects in Context. European Conference on Computer Vision (ECCV), 2014.
- HUGGING FACE. Spaces — Machine Learning Apps. Disponível em: <https://huggingface.co/spaces>. Acesso em: maio de 2026.